

DATABASE MANAGEMENT SYSTEM LABORATORY

LABORATORY MANUAL

IV Semester

(Academic Year: 2024-25)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Atria Institute of Technology
Bengaluru – 560 024

Title of the Course: Database Management Systems					Semester: IV	
Course Code :		BDBSB401		CIA Marks	50	
Course Type :		IPCC		SEE Marks	50	
Student Learning Hours (L:T:P:S)		42:0:28:56		Total Marks	100	
Credits		4		Exam Hours	03	
Examination Type (SEE)				Theory / Laboratory		
L - Lecture	T - Tutorial	P - Practical	S - Self Study	CL - Cognitive Level	PL - Psychomotor Level	
<u>PRACTICAL COMPONENTS OF IPCC</u>						
PART – A						
CONVENTIONAL EXPERIMENTS						
1.	Create a table called Employee & execute the following. Employee (EMPNO, ENAME, JOB, MANAGER_NO, SAL, COMMISSION) a. Create a user and grant all permissions to the user. b. Insert any three records in the employee table containing attributes EMPNO, ENAME, JOB, MANAGER_NO, SAL, COMMISSION and use rollback. Check the result c. Add primary key constraint and not null constraint to the employee table. d. Insert null values to the employee table and verify the result					
2.	Create a table called Employee that contains attributes Employee (EMPNO, ENAME, JOB, MGR, SALARY) and execute the following. a. Add a column commission with domain to the Employee table. b. Insert any five records into the table. c. Update the column details of job d. Rename the column of the Employee table using the alter command. e. Delete the employee whose Empno is 105.					
3.	Consider the following schema for Order Database: SALESMAN (Salesman_id, Name, City, Commission) CUSTOMER (Customer_id, Cust_Name, City, Grade, Salesman_id) ORDERS (Ord_No, Purchase_Amt, Ord_Date, Customer_id, Salesman_id) Write SQL queries to a. Count the customers with grades above Bangalore's average. b. Find the name and numbers of all salesmen who had more than one customer. c. List all the salesmen and indicate those who have and don't have customers in their cities (Use UNION operation.) d. Create a view that finds the salesman who has the customer with the highest order of a day. e. Demonstrate the DELETE operation by removing sales man with id 1000. All his orders must also be deleted.					
4.	Queries using aggregate functions (COUNT, AVG, MIN, MAX, SUM), Group by, Orderby. Employee (E_id, E_name, Age, Salary) a. Create an employee table containing all Records E_id, E_name, Age, Salary. b. Count number of employee names from employee table c. Find the Maximum age from the employee table. d. Find the Minimum age from the employee table. e. Find salaries of employees in Ascending g Order. f. Find grouped salaries of employees					
5.	Create a row level trigger for the customers table that would fire for INSERT or UPDATE or DELETE operations performed on the CUSTOMERS table. This trigger will display the salary difference between the old & new Salary. CUSTOMERS (ID, NAME, AGE, ADDRESS, SALARY)					

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Atria Institute of Technology
Bengaluru – 560 024

6.	Create cursor for Employee table & extract the values from the table. Declare the variables, Open the cursor & extract the values from the cursor. Close the cursor. Employee (E_id, E_name, Age, Salary)
7.	Install an Open-Source NoSQL Database MongoDB & perform basic CRUD (Create, Read, Update & Delete) operations. Execute MongoDB Basic Queries using CRUD operations

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Atria Institute of Technology
Bengaluru – 560 024

EXPERIMENT No. 01**Creation of Employee Table and Performing DDL & DML Operations in MySQL**

1.1 Objective	1.6 Observation
1.2 Apparatus Required	1.7 Results
1.3 Pre-Requisite	1.8 Discussions
1.4 Introduction	1.9 Pre-Requisite Question
1.5 Procedure	1.10 Post-Requisite

1.1 Objectives:

Create a table called Employee & execute the following. Employee (EMPNO, ENAME, JOB, MANAGER_NO, SAL, COMMISSION)

- Create a user and grant all permissions to the user.
- Insert any three records in the employee table containing attributes EMPNO, ENAME, JOB, MANAGER_NO, SAL, COMMISSION and use rollback. Check the result
- Add primary key constraint and not null constraint to the employee table.
- Insert null values to the employee table and verify the result

1.2 Apparatus Required:

- Computer System
- MySQL Server
- VS Code / MySQL Workbench

1.3 Pre-Requisite:

- Basic knowledge of SQL
- Understanding of DDL and DML commands
- Knowledge of constraints (Primary Key, NOT NULL)

1.4 Introduction:

SQL (Structured Query Language) is used to create and manage databases. DDL commands like CREATE, ALTER define structure. DML commands like INSERT, ROLLBACK manipulate data. Constraints ensure data integrity in tables.

1.5 Procedure:

```
CREATE TABLE Employee (
    EMPNO INT,
    ENAME VARCHAR(50),
    JOB VARCHAR(50),
    MANAGER_NO INT,
    SAL DECIMAL(10,2),
    COMMISSION DECIMAL(10,2)
);
desc Employee;
```

```
CREATE USER 'emp_user'@'localhost' IDENTIFIED BY '1234';
GRANT ALL PRIVILEGES ON USER_DB.* TO 'emp_user'@'localhost';
FLUSH PRIVILEGES;
```

```
INSERT INTO Employee VALUES (101, 'Ravi', 'Manager', 1001, 50000, 5000);
INSERT INTO Employee VALUES (102, 'Anu', 'Clerk', 1001, 25000, 2000);
INSERT INTO Employee VALUES (103, 'John', 'Analyst', 1002, 40000, 3000);
```

```
SELECT * FROM Employee;
```

```
ROLLBACK;
ALTER TABLE Employee
ADD CONSTRAINT emp_pk PRIMARY KEY (EMPNO);
```

```
ALTER TABLE Employee
MODIFY ENAME VARCHAR(50) NOT NULL;
```

```
ALTER TABLE Employee
MODIFY JOB VARCHAR(50) NOT NULL;
```

```
INSERT INTO Employee (EMPNO, ENAME, JOB)
VALUES (104, NULL, NULL)
```

1.6 Observation

- Records were inserted successfully.
- After ROLLBACK, inserted records were removed.
- Primary Key prevented duplicate EMPNO values.
- NOT NULL constraint prevented insertion of NULL values.

1.7 Results

Field	Type	Null	Key	Default	Extra
EMPNO	int	YES		null	
ENAME	varchar(50)	YES		null	
JOB	varchar(50)	YES		null	
MANAGER_NO	int	YES		null	
SAL	decimal(10,2)	YES		null	
COMMISSION	decimal(10,2)	YES		null	

EMPNO	ENAME	JOB	MANAGER_NO	SAL	COMMISSION
101	Ravi	Manager	1001	50000	5000
102	Anu	Clerk	1001	25000	2000
103	John	Analyst	1002	40000	3000

```
Error: ER_BAD_NULL_ERROR: Column 'ENAME' cannot be null
[Done] Finished MySQL query.
```

1.8 Discussions:

DDL commands define table structure. DML commands manipulate data. Primary Key ensures uniqueness. NOT NULL ensures mandatory fields. Rollback helps to undo uncommitted changes.

1.9 Pre – Experimentation Questions:

1. What is DDL?
2. What is DML?
3. What is Primary Key?
4. What is NOT NULL constraint?
5. What is ROLLBACK?

1.10 Post – Experimentation Questions :

Students learned:

- Creating database and tables
- Managing users and permissions
- Applying constraints
- Using transaction control (ROLLBACK)

EXPERIMENT No. 02**Creation and Manipulation of Employee Table Using SQL**

2.1 Objective	2.6 Observation
2.2 Apparatus Required	2.7 Results
2.3 Pre-Requisite	2.8 Discussions
2.4 Introduction	2.9 Pre-Requisite Question
2.5 Procedure	2.10 Post-Requisite

2.1 Objectives:

Create a table called Employee that contains attributes Employee (EMPNO, ENAME, JOB, MGR, SALARY) and execute the following.

- Add a column commission with domain to the Employee table.
- Insert any five records into the table.
- Update the column details of job
- Rename the column of the Employee table using the alter command.
- Delete the employee whose Empno is 105.

2.2 Apparatus Required:

- Computer System
- MySQL Server
- VS Code / MySQL Workbench

2.3 Pre-Requisite:

- Basic knowledge of DBMS
- Understanding of SQL commands:
 - CREATE
 - ALTER
 - INSERT
 - UPDATE
 - DELETE

2.4 Introduction:

Structured Query Language (SQL) is used to manage and manipulate relational databases. In this experiment, we create an **Employee** table and perform DDL (Data Definition Language) and DML (Data Manipulation Language) operations to understand how data is stored and modified in a database.

2.5 Procedure:

```
CREATE TABLE Employee (  
    EMPNO          INT,  
    ENAME          VARCHAR(50),  
    JOB            VARCHAR(50),  
    MGR            INT,  
    SALARY         DECIMAL(10,2)  
);
```

```
ALTER TABLE Employee ADD COMMISSION DECIMAL(10,2);
```

```
INSERT INTO Employee VALUES
```

```
(101, 'Ravi', 'Manager', 1001, 50000, 5000),
```

```
(102, 'Anu', 'Clerk', 1001, 25000, 2000),
```

```
(103, 'John', 'Analyst', 1002, 40000, 3000),
```

```
(104, 'Meena', 'HR', 1003, 35000, 1500),
```

```
(105, 'Arun', 'Salesman', 1001, 30000, 2500);
```

```
UPDATE Employee
```

```
SET JOB = 'Senior Clerk'
```

```
WHERE EMPNO = 102;
```

```
ALTER TABLE Employee
```

```
CHANGE SALARY SAL DECIMAL(10,2);
```

```
DELETE FROM Employee
```

```
WHERE EMPNO = 105;
```

2.6 Observation

- Table was successfully created.
- Commission column was added.
- Five records were inserted.
- Job of employee 102 was updated.
- Column SALARY was renamed to SAL.

- Employee with EMPNO 105 was deleted successfully.

2.7 Results

Field	Type	Null	Key	Default	Extra
EMPNO	int	YES		null	
ENAME	varchar(50)	YES		null	
JOB	varchar(50)	YES		null	
MGR	int	YES		null	
SALARY	decimal(10,2)	YES		null	

Field	Type	Null	Key	Default	Extra
EMPNO	int	YES		null	
ENAME	varchar(50)	YES		null	
JOB	varchar(50)	YES		null	
MGR	int	YES		null	
SALARY	decimal(10,2)	YES		null	
COMMISSION	decimal(10,2)	YES		null	

EMPNO	ENAME	JOB	MGR	SALARY	COMMISSION
101	Ravi	Manager	1001	50000	5000
102	Anu	Clerk	1001	25000	2000
103	John	Analyst	1002	40000	3000
104	Meena	HR	1003	35000	1500
105	Arun	Salesman	1001	30000	2500

EMPNO	ENAME	JOB	MGR	SALARY	COMMISSION
101	Ravi	Manager	1001	50000	5000
102	Anu	Senior Clerk	1001	25000	2000
103	John	Analyst	1002	40000	3000
104	Meena	HR	1003	35000	1500
105	Arun	Salesman	1001	30000	2500

EMPNO	ENAME	JOB	MGR	SAL	COMMISSION
101	Ravi	Manager	1001	50000	5000
102	Anu	Senior Clerk	1001	25000	2000
103	John	Analyst	1002	40000	3000
104	Meena	HR	1003	35000	1500
105	Arun	Salesman	1001	30000	2500

EMPNO	ENAME	JOB	MGR	SAL	COMMISSION
101	Ravi	Manager	1001	50000	5000
102	Anu	Senior Clerk	1001	25000	2000
103	John	Analyst	1002	40000	3000
104	Meena	HR	1003	35000	1500

2.8 Discussion

- CREATE TABLE defines table structure.
- ALTER TABLE modifies existing structure.
- INSERT adds records.
- UPDATE modifies existing records.
- DELETE removes records permanently.
- Renaming columns helps in restructuring the database without losing data.

2.9 Pre-Experimental Questions:

- What is DDL?
- What is DML?
- What is the difference between DELETE and DROP?
- What is a Primary Key?

2.10 Post-Experimental Questions:

- Practice writing more SQL queries.
- Learn constraints like NOT NULL, UNIQUE, CHECK.
- Perform SELECT queries with conditions.

EXPERIMENT No. 03**Advanced SQL Queries on Order Database**

3.1 Objective	3.6 Observation
3.2 Apparatus Required	3.7 Results
3.3 Pre-Requisite	3.8 Discussions
3.4 Introduction	3.9 Pre-Requisite Question
3.5 Procedure	3.10 Post-Requisite Question

3.1 Objectives:

Consider the following schema for Order Database:

SALESMAN (Salesman_id, Name, City, Commission)

CUSTOMER (Customer_id, Cust_Name, City, Grade, Salesman_id)

ORDERS (Ord_No, Purchase_Amt, Ord_Date, Customer_id, Salesman_id)

Write SQL queries to

- Count the customers with grades above Bangalore's average.
- Find the name and numbers of all salesmen who had more than one customer.
- List all the salesmen and indicate those who have and don't have customers in their cities (Use UNION operation.)
- Create a view that finds the salesman who has the customer with the highest order of a day.
- Demonstrate the DELETE operation by removing sales man with id 1000. All his orders must also be deleted.

3.2 Apparatus Required:

- Computer System
- MySQL Server
- VS Code / MySQL Workbench

3.3 Pre-Requisite:

- Basic knowledge of DBMS
- Understanding of:
 - SELECT statement
 - Aggregate functions (COUNT, AVG, MAX)
 - Subqueries
 - GROUP BY and HAVING
 - UNION
 - VIEW
 - DELETE

3.4 Introduction:

An Order Database consists of three related tables:

- **SALESMAN** (Salesman_id, Name, City, Commission)
- **CUSTOMER** (Customer_id, Cust_Name, City, Grade, Salesman_id)
- **ORDERS** (Ord_No, Purchase_Amt, Ord_Date, Customer_id, Salesman_id)

These tables are related using primary and foreign keys.

In this experiment, we perform advanced SQL operations such as subqueries, grouping, UNION, view creation, and cascading delete operations.

3.5 Procedure:

```
CREATE TABLE SALESMAN (
    Salesman_id INT PRIMARY KEY,
    Name VARCHAR(50),
    City VARCHAR(50),
    Commission DECIMAL(5,2)
);
```

```
CREATE TABLE CUSTOMER (
    Customer_id INT PRIMARY KEY,
    Cust_Name VARCHAR(50),
    City VARCHAR(50),
    Grade INT,
    Salesman_id INT,
    FOREIGN KEY (Salesman_id)
    REFERENCES SALESMAN(Salesman_id)
);
```

```
CREATE TABLE ORDERS (
    Ord_No INT PRIMARY KEY,
    Purchase_Amt DECIMAL(10,2),
    Ord_Date DATE,
    Customer_id INT,
    Salesman_id INT,
    FOREIGN KEY (Customer_id)
    REFERENCES CUSTOMER(Customer_id),
    FOREIGN KEY (Salesman_id)
    REFERENCES SALESMAN(Salesman_id)
    ON DELETE CASCADE
);
FOREIGN KEY (Salesman_id) REFERENCES SALESMAN(Salesman_id)
ON DELETE CASCADE
```

```
INSERT INTO SALESMAN VALUES
(1000, 'Ravi', 'Bangalore', 10.5),
(1001, 'Anil', 'Chennai', 12.0),
(1002, 'Meena', 'Bangalore', 11.0),
(1003, 'John', 'Mumbai', 9.5);
INSERT INTO CUSTOMER VALUES
(2001, 'Amit', 'Bangalore', 100, 1000),
(2002, 'Sneha', 'Chennai', 200, 1001),
(2003, 'Rahul', 'Bangalore', 150, 1000),
```

```
(2004, 'Priya', 'Mumbai', 300, 1003),  
(2005, 'Kiran', 'Bangalore', 250, 1002);
```

```
INSERT INTO ORDERS VALUES  
(3001, 5000, '2024-01-10', 2001, 1000),  
(3002, 7000, '2024-01-10', 2002, 1001),  
(3003, 4000, '2024-01-11', 2003, 1000),  
(3004, 9000, '2024-01-11', 2004, 1003),  
(3005, 6000, '2024-01-12', 2005, 1002);
```

```
SELECT COUNT(*)  
FROM CUSTOMER  
WHERE Grade > (  
    SELECT AVG(Grade)  
    FROM CUSTOMER  
    WHERE City = 'Bangalore'  
);
```

```
SELECT S.Name, COUNT(C.Customer_id) AS Total_Customers  
FROM SALESMAN S  
JOIN CUSTOMER C  
ON S.Salesman_id = C.Salesman_id  
GROUP BY S.Salesman_id, S.Name  
HAVING COUNT(C.Customer_id) > 1;
```

```
-- Salesmen who have customers  
SELECT S.Name, S.City, 'Has Customer' AS Status  
FROM SALESMAN S  
JOIN CUSTOMER C  
ON S.Salesman_id = C.Salesman_id
```

UNION

```
-- Salesmen who do not have customers  
SELECT S.Name, S.City, 'No Customer' AS Status  
FROM SALESMAN S  
WHERE S.Salesman_id NOT IN (  
    SELECT Salesman_id FROM CUSTOMER  
);
```

```
CREATE VIEW Highest_Order_Salesman AS  
SELECT S.Name, O.Ord_Date, MAX(O.Purchase_Amt) AS Highest_Order  
FROM ORDERS O  
JOIN SALESMAN S  
ON O.Salesman_id = S.Salesman_id  
GROUP BY O.Ord_Date, S.Name;
```

```
DELETE FROM ORDERS  
WHERE Salesman_id = 1000;
```

```
DELETE FROM SALESMAN WHERE Salesman_id = 1000;
```

3.6 Observation:

- Subqueries successfully calculated Bangalore's average grade.
- GROUP BY and HAVING identified salesmen with multiple customers.
- UNION operation displayed both categories of salesmen.
- View was created successfully for highest daily order.
- DELETE operation removed the salesman and corresponding orders.

3.7 Result:

Salesman_id	Name	City	Commission
1000	Ravi	Bangalore	10.5
1001	Anil	Chennai	12
1002	Meena	Bangalore	11
1003	John	Mumbai	9.5

Customer_id	Cust_Name	City	Grade	Salesman_id
2001	Amit	Bangalore	100	1000
2002	Sneha	Chennai	200	1001
2003	Rahul	Bangalore	150	1000
2004	Priya	Mumbai	300	1003
2005	Kiran	Bangalore	250	1002

Ord_No	Purchase_Amt	Ord_Date	Customer_id	Salesman_id
3001	5000	Wed Jan 10 2024 00:00:00 GMT+0530 (India Standard Time)	2001	1000
3002	7000	Wed Jan 10 2024 00:00:00 GMT+0530 (India Standard Time)	2002	1001
3003	4000	Thu Jan 11 2024 00:00:00 GMT+0530 (India Standard Time)	2003	1000
3004	9000	Thu Jan 11 2024 00:00:00 GMT+0530 (India Standard Time)	2004	1003
3005	6000	Fri Jan 12 2024 00:00:00 GMT+0530 (India Standard Time)	2005	1002

COUNT(*)

3

Name	Total_Customers
Ravi	2

Name	City	Status
Ravi	Bangalore	Has Customer
Anil	Chennai	Has Customer
Meena	Bangalore	Has Customer
John	Mumbai	Has Customer

3.8 Discussions:

- Subqueries help in comparing values with calculated results.
- GROUP BY groups records, HAVING filters grouped data.
- UNION combines results of two SELECT statements.
- VIEW stores a virtual table for complex queries.
- DELETE removes records permanently and must be handled carefully in related tables.

3.9 Pre – Experimentation Questions:

1. What is a subquery?
2. What is the difference between WHERE and HAVING?
3. What is UNION?
4. What is a View?
5. What is ON DELETE CASCADE?

3.10 Post – Experimentation Questions:

- Practice writing JOIN queries.
- Learn about indexing and performance tuning.
- Study triggers and stored procedures for advanced database management.

EXPERIMENT No. 04

SQL Queries Using Aggregate Functions, GROUP BY and ORDER BY

1.1 Objective	4.6 Observation
1.2 Apparatus Required	4.7 Results
1.3 Pre-Requisite	4.8 Discussions
1.4 Introduction	4.9 Pre-Requisite Question
1.5 Procedure	4.10 Post-Requisite

4.1 Objectives:

Queries using aggregate functions (COUNT, AVG, MIN, MAX, SUM), Group by, Orderby. Employee (E_id, E_name, Age, Salary)

- a. Create an employee table containing all Records E_id, E_name, Age, Salary.
- b. Count number of employee names from employee table
- Find the Maximum age from the employee table.
- d. Find the Minimum age from the employee table.
- e. Find salaries of employees in Ascending g Order.
- Find grouped salaries of employees

4.2 Apparatus Required:

- Computer System
- MySQL Server
- VS Code / MySQL Workbench

4.3 Pre-Requisite:

Basic knowledge of DBMS

- Understanding of SQL commands:
- CREATE
- INSERT
- SELECT
- Aggregate functions
- GROUP BY
- ORDER BY

4.4 Introduction:

Aggregate functions in SQL are used to perform calculations on multiple rows of a table and return a single value. Common aggregate functions include:

- COUNT() – Counts rows
- AVG() – Finds average
- MIN() – Finds minimum value
- MAX() – Finds maximum value

- SUM() – Finds total

In this experiment, we create an Employee table and apply aggregate functions along with GROUP BY and ORDER BY clauses.

4.5 Procedure:

```
CREATE TABLE Employee (E_id INT PRIMARY KEY,E_name VARCHAR(50),  
Age INT,Salary DECIMAL(10,2));
```

```
INSERT INTO Employee VALUES
```

```
(1, 'Ravi', 25, 30000),
```

```
(2, 'Anu', 30, 40000),
```

```
(3, 'John', 28, 35000),
```

```
(4, 'Meena', 35, 50000),
```

```
(5, 'Arun', 30, 40000);
```

```
SELECT COUNT(E_name) AS Total_Employees
```

```
FROM Employee;
```

```
SELECT MAX(Age) AS Maximum_Age
```

```
FROM Employee;
```

```
SELECT MIN(Age) AS Minimum_Age
```

```
FROM Employee;
```

```
SELECT E_name, Salary
```

```
FROM Employee
```

```
ORDER BY Salary ASC;
```

```
SELECT Salary, COUNT(*) AS Number_of_Employees
```

```
FROM Employee
```

```
GROUP BY Salary;
```

4.6 Observation:

- Employee table was successfully created.
- Aggregate functions returned correct calculated values.
- ORDER BY sorted salaries in ascending order.
- GROUP BY grouped employees based on salary values.

4.7 Results:

E_id	E_name	Age	Salary
1	Ravi	25	30000
2	Anu	30	40000
3	John	28	35000
4	Meena	35	50000
5	Arun	30	40000

Total_Employees
5

Maximum_Age
35

Minimum_Age
25

E_name	Salary
Ravi	30000
John	35000
Anu	40000
Arun	40000
Meena	50000

Salary	Number_of_Employees
30000	1
40000	2
35000	1
50000	1

4.8 Discussions:

- COUNT() counts number of records.
- MAX() and MIN() identify highest and lowest values.
- AVG() and SUM() perform mathematical calculations.
- ORDER BY sorts records.
- GROUP BY groups rows having the same values.

These functions are very useful in data analysis and report generation.

4.9 Pre – Experimentation Questions:

- What is an aggregate function?
- What is the difference between WHERE and GROUP BY?
- What is ORDER BY used for? What is the difference between COUNT(*) and COUNT(column_name)?

4.10 Post – Experimentation Questions:

- Practice writing JOIN queries.
- Learn about indexing and performance tuning.
- Study triggers and stored procedures for advanced database management.